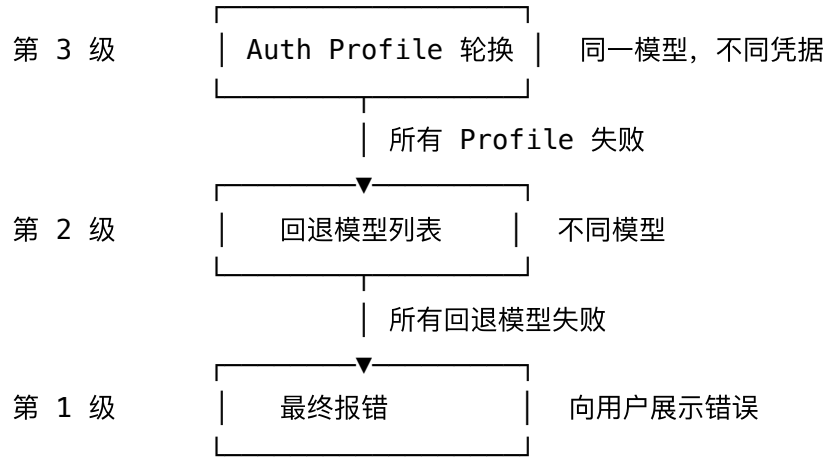
16.1 模型选择机制

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~400,000 tokens，输出 ~32,000 tokens（本章合计）

OpenClaw 支持数十个 LLM 提供者和上百个模型。用户发送一条消息时，系统需要决定：用哪个提供者？哪个模型？如果这个模型不可用怎么办？这就是**模型选择与故障转移**机制要解决的问题。OpenClaw 实现了一套三级容错策略——主模型、回退模型、和提供者内部的 Auth Profile 轮换。

16.1.1 主模型 → 回退模型 → 提供者内部 auth failover 的三级策略

OpenClaw 的模型容错分为三个层级：



第 3 级（最先尝试）——Auth Profile 轮换：同一个提供者可能配置了多个认证凭据（例如两个 Anthropic API Key）。当一个凭据因速率限制或认证错误失败时，系统自动切换到下一个可用凭据。这发生在 `runEmbeddedPiAgent` 内部的 `while(true)` 循环中（参见第 7.2 节）。

第 2 级——回退模型列表：如果一个模型的所有 Auth Profile 都失败了（或该模型本身不可用），系统抛出 `FailoverError`，由 `runWithModelFallback` 捕获并尝试列表中的下一个模型。

第 1 级——最终报错：如果所有回退模型也都失败了，系统将错误汇总并报告给用户。

16.1.2 模型选择源码（ src/agents/model-selection.ts ）

模型选择的核心职责是将用户配置（如 "anthropic/claude-sonnet-4-20250514" ）解析为标准化的提供者 + 模型 ID 对。

ModelRef 类型

```
// src/agents/model-selection.ts
export type ModelRef = {
  provider: string;  // 提供者标识（如 "anthropic"）
  model: string;     // 模型标识（如 "claude-sonnet-4-20250514"）
};
```

提供者 ID 标准化

不同来源的提供者名称可能有变体。 normalizeProviderId 将它们统一：

```
// src/agents/model-selection.ts
export function normalizeProviderId(provider: string): string {
  const normalized = provider.trim().toLowerCase();
  if (normalized === "z.ai" || normalized === "z-ai") return "zai";
  if (normalized === "opencode-zen") return "opencode";
  if (normalized === "qwen") return "qwen-portal";
  if (normalized === "kimi-code") return "kimi-coding";
  return normalized;
}
```

模型引用解析

用户可以用 provider/model 格式或纯模型名来引用模型：

```ts
// src/agents/model-selection.ts
export function parseModelRef(raw: string, defaultProvider: string): ModelRef | null {
  const trimmed = raw.trim();
  if (!trimmed) return null;

  const slash = trimmed.indexOf("/");
  if (slash === -1) {
    // 无 "/" 则使用默认提供者
    const provider = normalizeProviderId(defaultProvider);
    const model = normalizeProviderModelId(provider, trimmed);
    return { provider, model };
  }

  // 有 "/" 则拆分
  const provider = normalizeProviderId(trimmed.slice(0, slash));
  const model = normalizeProviderModelId(provider, trimmed.slice(slash + 1));
  return { provider, model };
}
```

模型别名

Anthropic 的模型支持简短别名：

```ts
const ANTHROPIC_MODEL_ALIASES: Record<string, string> = {
  "opus-4.6": "claude-opus-4-6",
  "opus-4.5": "claude-opus-4-5",
  "sonnet-4.5": "claude-sonnet-4-5",
};
```

用户可以用 `opus-4.6` 代替完整的 `claude-opus-4-6`，系统自动展开。

模型允许列表

管理员可以配置允许使用的模型列表，限制用户可以通过 `/model` 指令切换到的模型范围：

```ts
// src/agents/model-selection.ts
export function buildAllowedModelSet(params: {
  cfg: OpenClawConfig;
  catalog: ModelCatalogEntry[];
  defaultProvider: string;
  defaultModel: string;
}): { allowedKeys: Set<string>; allowedCatalog: ModelCatalogEntry[] } {
  // 从 agents.defaults.models 配置构建允许列表
  const configuredKeys = buildConfiguredAllowlistKeys({ cfg, defaultProvider });
  // 默认模型始终在允许列表中
  // 目录中与允许列表匹配的模型可被选择
}
```

CLI 提供者检测

某些提供者需要以独立 CLI 进程运行（而非嵌入模式）：

```ts
// src/agents/model-selection.ts
export function isCliProvider(provider: string, cfg?: OpenClawConfig): boolean {
  const normalized = normalizeProviderId(provider);
  if (normalized === "claude-cli" || normalized === "codex-cli") return true;
  // 也支持自定义 CLI 后端
  const backends = cfg?.agents?.defaults?.cliBackends ?? {};
  return Object.keys(backends).some((key) => normalizeProviderId(key) === normalized);
}
```

16.1.3 模型回退（ `src/agents/model-fallback.ts` ）

模型回退是三级容错的第二级。 `runWithModelFallback` 函数管理一个有序的候选模型列表，依次尝试直到某个模型成功。

候选列表构建

```ts
// src/agents/model-fallback.ts（简化版）
function resolveFallbackCandidates(params: {
  cfg: OpenClawConfig;
  provider: string;
  model: string;
  fallbacksOverride?: string[];
}): ModelCandidate[] {
  const candidates: ModelCandidate[] = [];

  // 1. 首选模型（当前请求的模型）
  candidates.push({ provider: params.provider, model: params.model });

  // 2. 配置的回退模型列表
  const modelFallbacks = params.fallbacksOverride
    ?? params.cfg.agents?.defaults?.model?.fallbacks
    ?? [];
  for (const raw of modelFallbacks) {
    candidates.push(resolveModelRefFromString(raw));
  }

  // 3. 全局默认模型（作为最终兜底）
  if (!params.fallbacksOverride) {
    const primary = resolveConfiguredModelRef({ cfg: params.cfg });
    candidates.push({ provider: primary.provider, model: primary.model });
  }

  return candidates; // 去重后返回
}
```

候选列表的顺序决定了尝试优先级：首选模型 → 回退模型 1 → 回退模型 2 → ... → 全局默认模型。

回退执行循环

```
// src/agents/model-fallback.ts
export async function runWithModelFallback<T>(params: {
  cfg: OpenClawConfig;
  provider: string;
  model: string;
  agentDir?: string;
  fallbacksOverride?: string[];
  run: (provider: string, model: string) => Promise<T>;
}): Promise<{ result: T; provider: string; model: string; attempts: FallbackAttempt[] }
  const candidates = resolveFallbackCandidates({ ... });
  const attempts: FallbackAttempt[] = [];

  for (let i = 0; i < candidates.length; i++) {
    const candidate = candidates[i];

    // 跳过所有 Profile 都在冷却期的提供者
    if (authStore) {
      const profileIds = resolveAuthProfileOrder({ store: authStore, provider: candidat
      const anyAvailable = profileIds.some((id) => !isProfileInCooldown(authStore, id))
      if (profileIds.length > 0 && !anyAvailable) {
        attempts.push({ ...candidate, error: "Provider in cooldown", reason: "rate_limi
        continue;  // 跳过，不尝试
      }
    }

    try {
      const result = await params.run(candidate.provider, candidate.model);
      return { result, provider: candidate.provider, model: candidate.model, attempts }
    } catch (err) {
      // AbortError（用户主动取消）不应触发回退
      if (shouldRethrowAbort(err)) throw err;

      // 非 FailoverError 不应触发回退
      if (!isFailoverError(coerceToFailoverError(err, candidate))) throw err;

      attempts.push({ ...candidate, error: err.message, reason: err.reason });
    }
  }

  // 所有候选都失败
```

```
  throw new Error(`All models failed (${attempts.length}): ${summary}`);
}
```

几个关键设计决策：

1. **冷却期预检查**——如果某个提供者的所有 Auth Profile 都在冷却期，直接跳过，不浪费时间尝试
2. **AbortError 穿透**——用户主动取消不应触发回退，应立即中止整个流程
3. **非 FailoverError 穿透**——只有明确标记为"可故障转移"的错误才触发回退，其他错误（如配置错误、编程错误）直接上抛
4. **错误聚合**——所有失败尝试的详情被收集，最终报错时可以展示完整的失败链

16.1.4 模型兼容性层（`src/agents/model-compat.ts`）

不同的 LLM 提供者对 API 的支持程度不同。例如，OpenAI 的 API 格式被广泛用作“行业标准”，许多第三方提供者（如 Z.AI）提供了兼容 OpenAI API 的接口，但并不支持所有特性。

```
// src/agents/model-compat.ts
export function normalizeModelCompat(model: Model<Api>): Model<Api> {
  const isZai = model.provider === "zai" || model.baseUrl?.includes("api.z.ai");
  if (!isZai || model.api !== "openai-completions") return model;

  // Z.AI 不支持 developer role，需要在兼容性层禁用
  const compat = model.compat ?? {};
  model.compat = { ...compat, supportsDeveloperRole: false };
  return model;
}
```

> **衍生解释**：OpenAI 的 Chat Completions API 定义了多种消息角色：`system`（系统提示）、`user`（用户消息）、`assistant`（助手回复）和较新的 `developer`（开发者指令）。`developer` 角色是 OpenAI 在 2024 年引入的，用于替代 `system` 角色中的开发者级指令。但许多兼容 OpenAI API 的第三方提供者尚未支持 `developer` 角色，发送包含该角色的请求会导致错误。`normalizeModelCompat` 通过在兼容性配置中禁用不支持的特性来避免这类问题。

模型兼容性层目前较为简洁，但它提供了一个清晰的扩展点——随着更多提供者和模型的加入，可以在此添加更多的兼容性适配逻辑。

本节小结

1. **三级容错策略**：Auth Profile 轮换（同模型不同凭据）→ 回退模型列表（不同模型）→ 最终报错，层层递进确保服务可用性。
2. **模型选择**通过标准化提供者 ID 和模型 ID、支持别名、维护允许列表等机制，将灵活的用户配置映射到精确的模型引用。
3. **模型回退**维护有序的候选列表，跳过冷却期提供者，区分可回退错误和不可回退错误，聚合所有失败尝试的详情。
4. **模型兼容性层**处理不同提供者对 API 标准支持的差异，确保请求格式与目标提供者兼容。